

SENA – Formación Tecnológica

ACTIVIDAD PRÁCTICA

Implementación de TLS/SSL en Vue.js con Vite

Proyecto: Texticode – Sistema de Gestión Textil

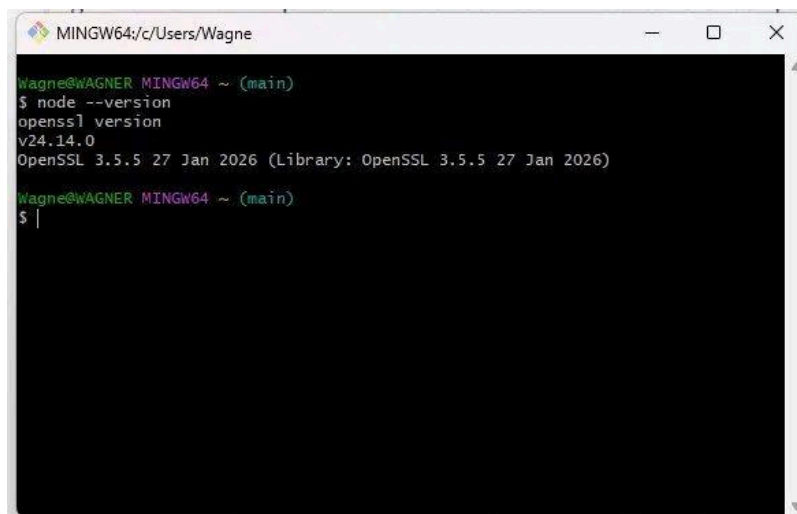
Estudiantes: Kevin Wagner - Andres Lancheros - Camilo Tibambre - Kevin Castro

Instructor: Ing. Edwin Marín

Fecha: 15 de junio de 2026

Paso 1 – Verificación de herramientas

Se verificó que Node.js y OpenSSL estuvieran correctamente instalados en el sistema. En Windows se utilizó Git Bash, ya que OpenSSL no está disponible en PowerShell por defecto.



```
MINGW64~/c/Users/Wagne
Wagne@WAGNER MINGW64 ~ (main)
$ node --version
v24.14.0
$ openssl version
OpenSSL 3.5.5 27 Jan 2026 (Library: OpenSSL 3.5.5 27 Jan 2026)
Wagne@WAGNER MINGW64 ~ (main)
$ |
```

Figura 1 – Node.js v24.14.0 y OpenSSL 3.5.5 verificados en Git Bash

Paso 2 – Generación del certificado autofirmado

Se creó la carpeta certs dentro del frontend y se generó un certificado X.509 autofirmado con OpenSSL con los datos del proyecto: Bogotá, SENA, Texticode y dominio localhost, con validez de 365 días.

Nota: Se requirió agregar MSYS_NO_PATHCONV=1 porque Git Bash en Windows interpreta /C=CO como una ruta del disco C:, generando un error. Esta variable deshabilita esa conversión automática.

```
$ mkdir certs
$ MSYS_NO_PATHCONV=1 openssl req -x509 -newkey rsa:2048 -nodes \
  -keyout certs/localhost.key -out certs/localhost.crt -days 365 \
  -subj "/C=CO/ST=Bogota/L=Bogota/O=SENA/OU=Texticode/CN=localhost"
```

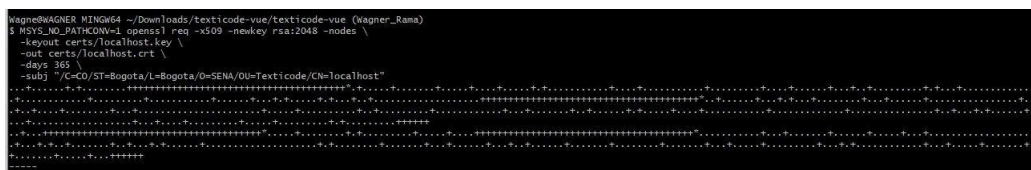


Figura 2 – Generación exitosa del certificado y la llave privada

Paso 3 – Verificación del certificado con OpenSSL

Se inspeccionó el certificado generado para confirmar los datos y registrar la fecha de vencimiento en la bitácora.

```
$ openssl x509 -in certs/localhost.crt -text -noout | grep -E "Subject:|Not After"
```

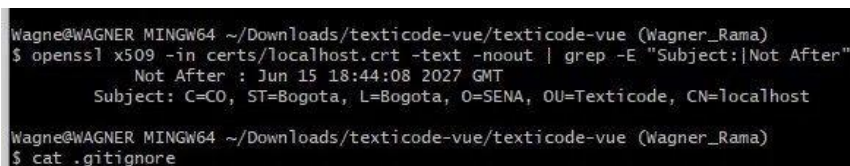


Figura 3 – Subject y fecha de expiración confirmados

Registro de bitácora:

Subject: C=CO, ST=Bogota, L=Bogota, O=SENA, OU=Texticode, CN=localhost

Not After: 15 de junio de 2027 (365 días de validez)

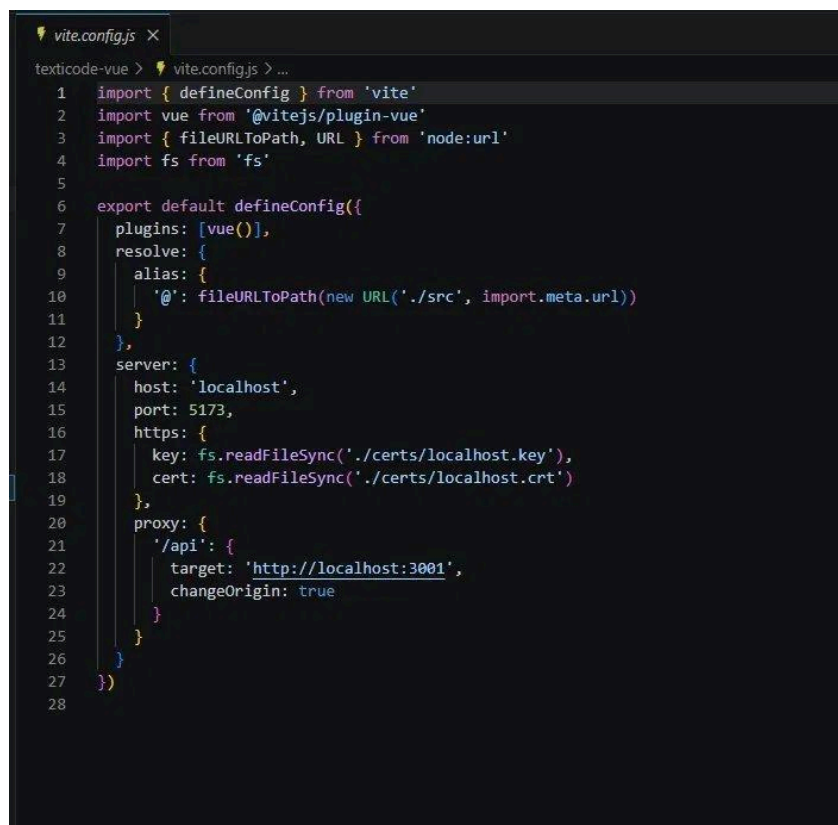
Algoritmo: RSA 2048 bits – certificado X.509 autofirmado

Si se regenerara con -days 60, la fecha de expiración sería aproximadamente el 14 de agosto de 2026. Una validez menor obliga a renovar el certificado más frecuentemente, pero reduce el tiempo de exposición si la llave privada se ve comprometida.

Paso 4 – Configuración de archivos del proyecto

4.1 vite.config.js

Se modificó vite.config.js para habilitar HTTPS usando fs.readFileSync, cargando el certificado y la llave privada. Se conservó el proxy hacia el backend Express en el puerto 3001.

A screenshot of a code editor showing the vite.config.js file. The code is written in JavaScript and configures Vite for a Vue project. It imports defineConfig from vite, vue from @vitejs/plugin-vue, fileURLToPath and URL from node:url, and fs from fs. The default export is a function that returns an object with plugins, resolve, server, and proxy configurations. The server configuration includes host, port, and https options with fs.readFileSync for key and cert. The proxy configuration maps /api to http://localhost:3001.

```
1 import { defineConfig } from 'vite'
2 import vue from '@vitejs/plugin-vue'
3 import { fileURLToPath, URL } from 'node:url'
4 import fs from 'fs'
5
6 export default defineConfig({
7   plugins: [vue()],
8   resolve: {
9     alias: {
10       '@': fileURLToPath(new URL('./src', import.meta.url))
11     }
12   },
13   server: {
14     host: 'localhost',
15     port: 5173,
16     https: {
17       key: fs.readFileSync('./certs/localhost.key'),
18       cert: fs.readFileSync('./certs/localhost.crt')
19     },
20     proxy: {
21       '/api': {
22         target: 'http://localhost:3001',
23         changeOrigin: true
24       }
25     }
26   }
27 })
```

Figura 4 – vite.config.js configurado con HTTPS y proxy al backend

4.2 .gitignore

Se agregó la carpeta certs/ al .gitignore para que los archivos .key y .crt no se suban al repositorio. Subir la llave privada representaría un riesgo crítico de seguridad pues cualquiera podría hacerse pasar por el servidor.

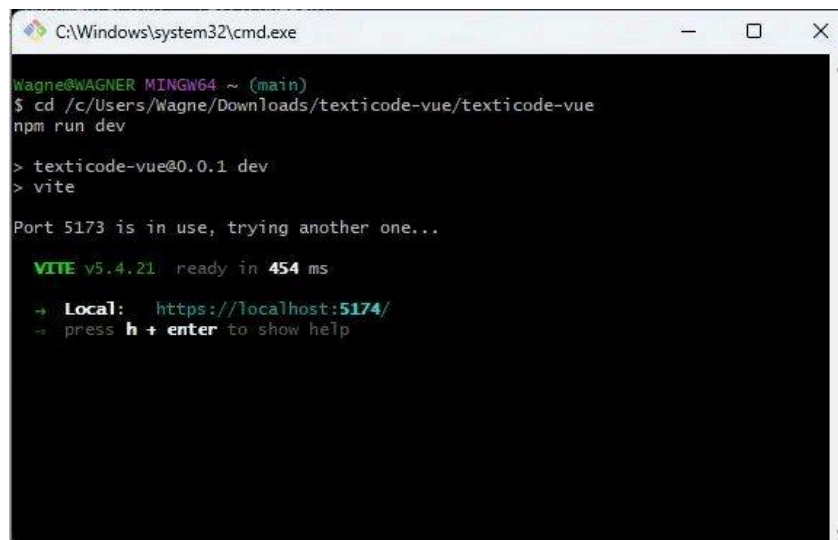
```
# Certificados dentro del frontend
texticode-vue/certs/*.key
texticode-vue/certs/*.crt
```

4.3 README.md

Se actualizó el README.md con el procedimiento completo. Como los certificados no se suben al repo, cualquier desarrollador que clone el proyecto necesita saber que debe generarlos localmente con los comandos documentados.

Paso 5 – Ejecución del servidor en modo HTTPS

Con el certificado generado y el vite.config.js configurado, se levantó el servidor de desarrollo. Vite reportó la URL segura en consola confirmando que HTTPS quedó activo.



```
C:\Windows\system32\cmd.exe
Wagne@WAGNER MINGW64 ~ (main)
$ cd /c/Users/Wagne/Downloads/texticode-vue/texticode-vue
npm run dev

> texticode-vue@0.0.1 dev
> vite

Port 5173 is in use, trying another one...

VITE v5.4.21 ready in 454 ms
➔ Local:   https://localhost:5174/
➔ press h + enter to show help
```

Figura 5 – Vite v5.4.21 listo en https://localhost:5174/

Paso 6 – Prueba en el navegador

6.1 Advertencia del certificado autofirmado

Al acceder a https://localhost:5174 Chrome muestra el error NET::ERR_CERT_AUTHORITY_INVALID. Este es el comportamiento esperado: el certificado es autofirmado y Chrome no puede verificar la identidad del servidor con ninguna CA reconocida.

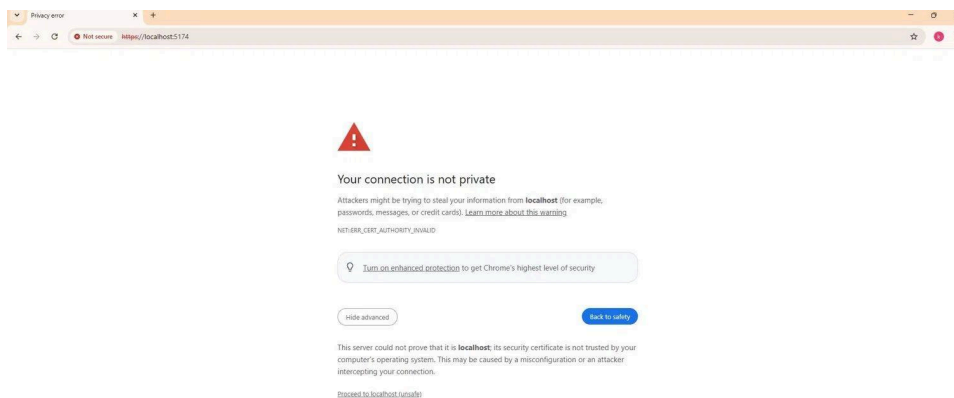


Figura 6 – Advertencia de privacidad en Chrome por certificado autofirmado

6.2 Proyecto cargando en HTTPS

Al hacer clic en "Proceed to localhost (unsafe)" el proyecto Texticode carga correctamente bajo HTTPS. La URL en la barra del navegador confirma la conexión cifrada activa.

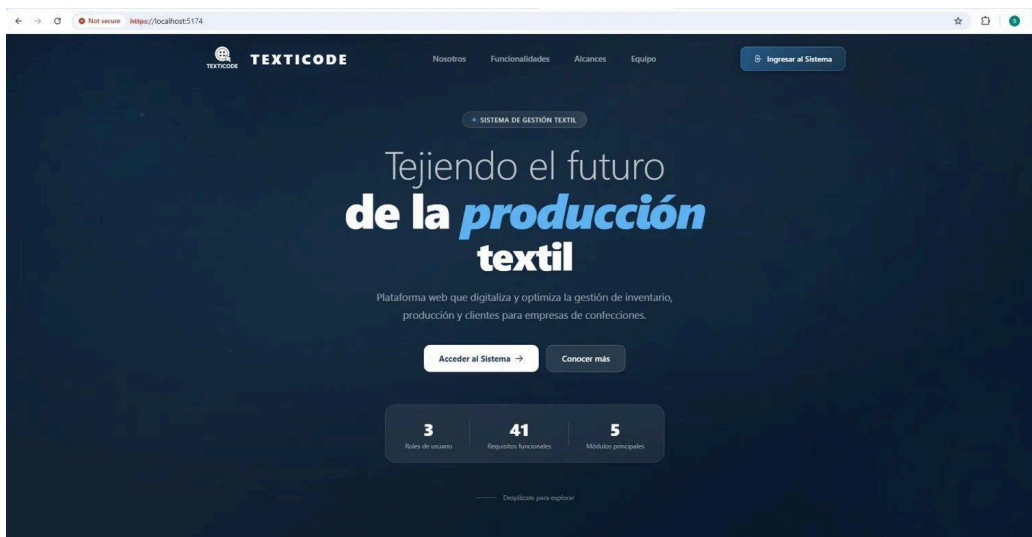


Figura 7 – Texticode ejecutándose correctamente en `https://localhost:5174`

Riesgos del certificado autofirmado

- El navegador muestra advertencia de privacidad porque no hay una CA que respalde la identidad del servidor.
- No es apto para producción: los usuarios finales no deberían aceptar excepciones de seguridad manualmente.
- Vulnerable a ataques Man-in-the-Middle (MITM): un atacante puede crear un certificado falso con los mismos datos sin que el cliente lo detecte.
- Sin mecanismo de revocación: si la llave privada se compromete no se puede invalidar el certificado antes de su vencimiento.
- Uso exclusivo para desarrollo local – válido únicamente en el entorno controlado del desarrollador.

Conclusiones

Se implementó exitosamente HTTPS en el proyecto Texticode usando Vite y un certificado TLS autofirmado con OpenSSL. Los datos del certificado quedaron correctamente configurados: ST=Bogota, L=Bogota, O=SENA, OU=Texticode, CN=localhost, con validez de 365 días. El ejercicio permitió comprender el protocolo TLS/SSL, la estructura X.509, el rol de las Autoridades Certificadoras y la diferencia entre desarrollo y producción en materia de seguridad. Para producción se recomienda usar Let's Encrypt, que es gratuito y reconocido por todos los navegadores.